École Centrale de Lille

Projet d'Intégration G3
Parcours SDI

Contrôle Thermique CVC par Apprentissage par Renforcement Multi-Agents

Eleve :
Soufiane Najari

Encadrant :
Pascal YIM

6 mars 2026

Table des matières

1 Introduction et Contexte **3**

2 Modélisation Physique et Environnement **3**
2.1 Modélisation Physique : l'Analogie Résistance-Capacité (RC) 3
2.1.1 Estimation des paramètres : Résistance thermique 4
2.1.2 Discrétisation et Résolution Numérique 4
2.2 Environnement Multi-Agents (PettingZoo) 4
2.2.1 Topologie du bâtiment simulé 5
2.2.2 Espace d'observation 5
2.2.3 Espace d'action 5
2.2.4 Fonction de récompense 5

3 Principes de l'Apprentissage par Renforcement (RL) **7**
3.1 Le Processus de Décision de Markov (MDP) 7
3.2 Formalisation Multi-Agents (MARL) 7
3.3 De PPO à IPPO : l'architecture Multi-Agents retenue 7
3.3.1 Le réseau de neurones comme politique : MlpPolicy 7
3.3.2 L'architecture Actor-Critic 8
3.3.3 L'avantage : mesurer si une action était bonne ou mauvaise 8
3.3.4 Le clipping PPO : pourquoi ne pas simplement faire de la descente de gradient ? 9
3.3.5 Le cycle d'entraînement dans notre implémentation 9
3.3.6 Pourquoi PPO et pas un autre algorithme 10
3.3.7 IPPO : ce que produit cette architecture 10

4 Baselines de Contrôle Classique **11**
4.1 Régulateur Proportionnel (P) 11
4.2 Régulateur Proportionnel-Intégral (PI) 11
4.3 Limites des approches classiques 11

5 Méthodologie et Implémentation **13**
5.1 Prise en main des outils 13
5.2 Développement du modèle physique 13
5.3 Développement de l'environnement et entraînement 13
5.4 Implémentation des baselines et comparaison 13

6 Résultats et Analyse des Performances **14**
6.1 Protocole expérimental 14
6.2 Comparaison avec les Baselines 14
6.2.1 Scénario Estival 14
6.2.2 Scénario Hivernal 16
6.3 Indicateurs Clés de Performance (KPIs) 17
6.4 Mise en perspective : un modèle sans forecasting 17
6.5 Analyse du comportement : l'effet du Passager Clandestin (Free-riding) . . 18
6.6 Comportement qualitatif du PPO 18

7 Compétences Développées **19**

8 Conclusion **19**

Bibliographie **20**

1 Introduction et Contexte

Ce projet vise à développer un système de contrôle intelligent pour le chauffage, la ventilation et la climatisation (CVC) en utilisant l'apprentissage par renforcement multi-agents (MARL). L'objectif est de permettre à des agents autonomes de coopérer pour minimiser la consommation énergétique globale tout en maintenant le confort thermique des occupants.

L'innovation principale de ce projet réside dans l'hybridation entre une modélisation physique "White-box" ultra-rapide et des algorithmes d'IA avancés (MARL). Pour contourner la lourdeur computationnelle des simulateurs thermiques complexes de type "Black-box" (comme EnergyPlus [1]), nous avons créé un environnement de simulation rapide basé sur un modèle physique simplifié de type Résistance-Capacité (RC). Cette méthode permet un entraînement des agents sur des millions de pas de temps en quelques heures, rendant le déploiement du RL réaliste pour des applications en temps réel.

`Code source : https://github.com/SoufianeNajari/Reinforcement-Learning-Multi-Agent-Thermal-Control`

2 Modélisation Physique et Environnement

2.1 Modélisation Physique : l'Analogie Résistance-Capacité (RC)

Pour simuler le comportement thermique du bâtiment, nous utilisons une analogie électrique où les transferts de chaleur sont représentés par des circuits RC. Dans cette approche :

- Les **Capacités thermiques** (C en J/K) représentent l'inertie du bâtiment (capacité de l'air et des murs à stocker la chaleur).
- Les **Résistances thermiques** (R en K/W) représentent l'isolation des parois et les échanges entre les zones.

Le modèle implémenté est de type **multi-zones couplées**. Pour chaque zone i, l'évolution de la température T_i est régie par l'équation du bilan thermique :

$$C_i \frac{dT_i}{dt} = \frac{T_{ext} - T_i}{R_{ext}} + \sum_{j \in voisins} \frac{T_j - T_i}{R_{i,j}} + Q_{hvac,i} \tag{1}$$

Où Q_{hvac} représente l'apport (ou le retrait) d'énergie par le système de climatisation.

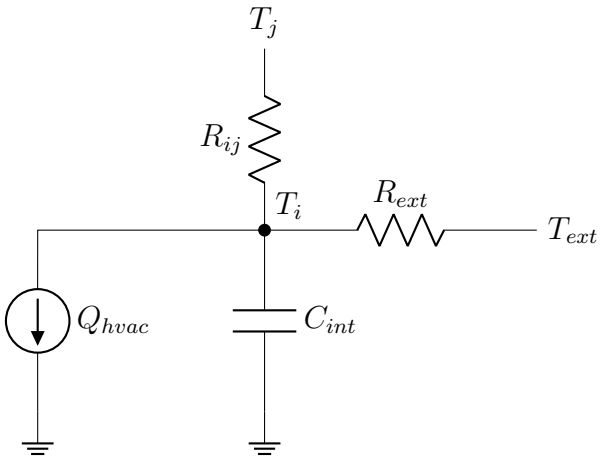

FIGURE 1 – Schéma du circuit électrique équivalent (RC) pour une zone i.

2.1.1 Estimation des paramètres : Résistance thermique

Pour estimer les valeurs de résistance R utilisées dans la matrice de transfert, nous nous sommes appuyés sur les caractéristiques géométriques et matérielles des parois selon la relation :

$$R = \frac{e}{\lambda \cdot S} \tag{2}$$

Où :
— e est l'épaisseur de la paroi (m).
— λ est la conductivité thermique du matériau ($W \cdot m^{-1} \cdot K^{-1}$).
— S est la surface d'échange (m^2).

Cette approche nous a permis de calibrer le modèle physique pour qu'il soit représentatif d'un bâtiment réel (ex : béton isolé par l'extérieur), garantissant que l'agent RL apprend sur des constantes de temps réalistes.

2.1.2 Discrétisation et Résolution Numérique

Pour intégrer ce système dans notre environnement de Reinforcement Learning, nous discrétisons l'équation différentielle via la **méthode d'Euler explicite** avec un pas de temps $\Delta t = 60s$:

$$T_i(t + \Delta t) = T_i(t) + \frac{\Delta t}{C_i} \left(\Phi_{ext} + \Phi_{inter-zones} + Q_{hvac,i} \right) \tag{3}$$

Cette formulation permet un calcul itératif extrêmement rapide, indispensable pour les algorithmes de MARL qui nécessitent des millions d'interactions pour converger.

2.2 Environnement Multi-Agents (PettingZoo)

Le modèle physique a été intégré dans un environnement compatible avec l'API **PettingZoo** (basée sur Gymnasium), instanciant trois agents correspondant aux trois zones thermiques du bâtiment simulé.

2.2.1 Topologie du bâtiment simulé

Le bâtiment est modélisé avec **3 zones** dont les relations sont décrites par la matrice d'adjacence suivante :

$$A = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{pmatrix} \tag{4}$$

Cette topologie correspond à une architecture en **étoile** : la zone 0 est une zone tampon centrale (couloir ou salle intérieure) sans exposition directe à l'extérieur ($e_0 = 0$), tandis que les zones 1 et 2 sont des pièces périphériques exposées ($e_1 = e_2 = 1$). Les zones 1 et 2 ne communiquent entre elles que via la zone 0.

Les offsets de température extérieure (ΔT_{ext}) reflètent les différences d'exposition selon l'orientation : zone 1 orientée nord ($-2°C$) et zone 2 orientée sud ($+3°C$).

2.2.2 Espace d'observation

Chaque agent i reçoit un vecteur d'observation de dimension 5, entièrement **centré autour de zéro** pour faciliter l'apprentissage :

Indice	Signal	Formule
0	Écart à la consigne	$T_i - T^*$
1	Gradient extérieur	$T_{ext,i} - T_i$
2	Gradient voisinage	$\bar{T}_{voisins} - T_i$
3	Prix de l'électricité	$p(t) \in \{0.15, 0.25\}$ €/kWh
4	Nombre de voisins	$\lvert\mathcal{N}_i\rvert$

TABLE 1 – Espace d'observation par agent.

2.2.3 Espace d'action

Chaque agent contrôle une action continue $a_i \in [-1, 1]$ représentant la commande normalisée du système CVC : -1 correspond à la climatisation à pleine puissance, $+1$ au chauffage à pleine puissance. La puissance physique est obtenue par $Q_{hvac,i} = a_i \times P_{max}$ avec $P_{max} = 2000$ W.

2.2.4 Fonction de récompense

La fonction de récompense est un compromis entre confort thermique et coût énergétique. Pour chaque agent i à chaque pas de temps :

$$r_i = -\frac{1}{20}\left[\alpha\left(|e_i| + e_i^2\right) + \beta \cdot \frac{|a_i| \cdot P_{max} \cdot p(t)}{60}\right] \tag{5}$$

Où $e_i = |T_i - T^*|$ est l'erreur de confort, $p(t)$ le prix de l'électricité, et $(\alpha, \beta) = (2.0,\ 1.0)$ les poids de l'arbitrage confort/énergie.

La pénalité d'inconfort utilise une formulation **Elastic Net** ($|e| + e^2$) : linéaire pour les petits écarts (sensibilité aux erreurs résiduelles) et quadratique pour les grands écarts (forte pénalisation des dépassements).

Le prix $p(t)$ suit une tarification **heures pleines** / **heures creuses** :

$$p(t) = \begin{cases} 0.15 \text{ €/kWh} & \text{si } t \notin [6h, 22h] \\ 0.25 \text{ €/kWh} & \text{sinon} \end{cases} \tag{6}$$

Le facteur de normalisation 1/20 ramène les récompenses dans une plage $[-1, 0]$, ce qui stabilise l'entraînement des algorithmes PPO.

3 Principes de l'Apprentissage par Renforcement (RL)

L'Apprentissage par Renforcement est une branche de l'intelligence artificielle où un **agent** apprend à prendre des décisions optimales par essais et erreurs en interagissant avec un **environnement**.

3.1 Le Processus de Décision de Markov (MDP)

Le fonctionnement du système repose sur une boucle itérative entre l'agent (le contrôleur CVC) et l'environnement (le bâtiment) :

— **État** (s_t) : Représente les observations à l'instant t (températures, météo, prix).
— **Action** (a_t) : La décision prise par l'agent (niveau de puissance de la PAC).
— **Récompense** (r_t) : Un signal scalaire indiquant la qualité de l'action.

L'objectif de l'agent est de maximiser l'espérance du gain cumulé, noté G_t :

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k} \tag{7}$$

Où γ est le facteur d'actualisation (discount factor), permettant de pondérer l'importance des récompenses futures par rapport aux récompenses immédiates.

3.2 Formalisation Multi-Agents (MARL)

Dans notre cas, chaque zone du bâtiment est pilotée par un agent distinct. Cette approche **Multi-Agents** permet une meilleure scalabilité. Les agents doivent apprendre une stratégie coopérative [4] : une zone peut choisir de "sur-chauffer" légèrement si cela permet de limiter les pertes thermiques d'une zone voisine plus sensible.

Concept RL	**Application CVC**
Agent	Régulateur intelligent de zone
Environnement	Modèle thermique RC
État (s)	Écart à la consigne, Delta extérieur, Prix de l'énergie
Action (a)	Commande continue $\in [-1, 1]$ (Climatisation / Chauffage)
Récompense (r)	Somme pondérée du confort et du coût énergétique

TABLE 2 – Correspondance entre les concepts de RL et le système CVC.

3.3 De PPO à IPPO : l'architecture Multi-Agents retenue

3.3.1 Le réseau de neurones comme politique : MlpPolicy

La politique π est la fonction centrale du RL : elle prend un état en entrée et retourne une action. Pour la représenter, il faut choisir une architecture mathématique.

Dans notre cas, l'état est un vecteur de 5 nombres réels continus. Une table de correspondance état $\rightarrow$ action est impossible (l'espace des états est infini). On utilise donc un **réseau de neurones**, qui est un approximateur universel capable de représenter n'importe quelle fonction continue.

`MlpPolicy` signifie **Multi-Layer Perceptron Policy** : la politique est un réseau de neurones à couches denses entièrement connectées. C'est le choix naturel quand l'entrée

est un vecteur numérique. Le réseau prend en entrée le vecteur d'observation de taille 5 et produit en sortie une **distribution gaussienne** $\mathcal{N}(\mu, \sigma)$ sur l'action continue $a_i \in [-1, 1]$. L'action réelle est tirée de cette distribution, ce qui assure l'exploration.

3.3.2 L'architecture Actor-Critic

PPO est un algorithme **Actor-Critic** [2] : il maintient deux réseaux aux rôles complémentaires.

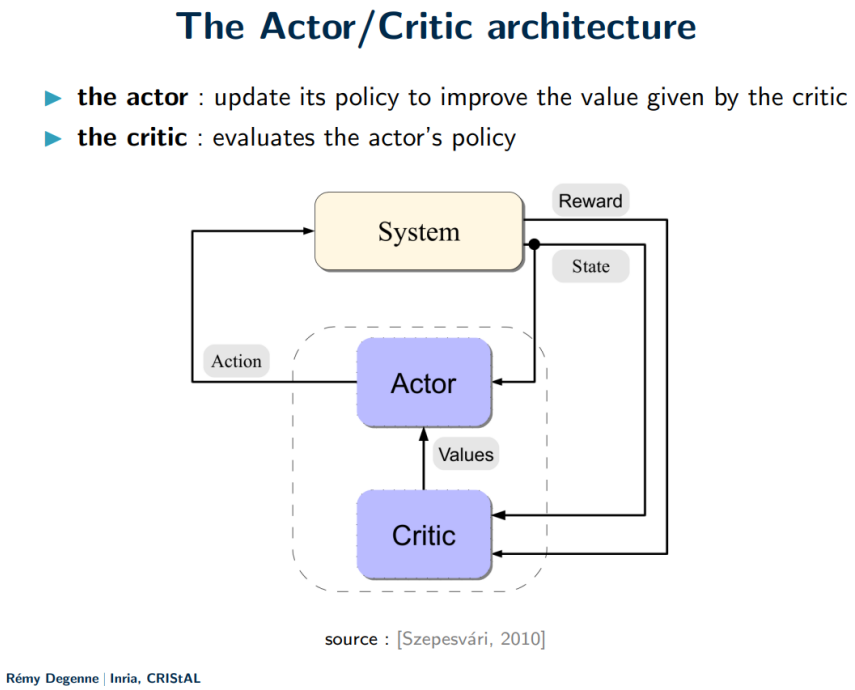

FIGURE 2 – Architecture Actor-Critic : l'Actor choisit l'action, le Critic évalue l'état, tous deux guidés par le signal de récompense de l'environnement.

L'Actor (le décideur) représente la politique π_θ. Il observe l'état s_t et produit une distribution sur les actions. C'est lui qui agit dans l'environnement.

Le Critic (l'évaluateur) estime la **fonction de valeur** $V(s_t)$: combien de récompense cumulée l'agent va-t-il accumuler à partir de l'état s_t en suivant sa politique actuelle ? Il ne choisit aucune action, il sert uniquement à guider la mise à jour de l'Actor.

Les deux réseaux partagent les premières couches (extraction de features communes) puis se séparent en deux têtes distinctes.

3.3.3 L'avantage : mesurer si une action était bonne ou mauvaise

Pour mettre à jour l'Actor, il faut savoir si l'action prise était **meilleure ou moins bonne que ce qui était attendu**. La récompense brute r_t seule ne suffit pas : une récompense de -0.3 peut être excellente si l'état était difficile, ou médiocre si l'état était favorable.

Le Critic permet de calculer l'**avantage** $\hat{A}_t$:

$$\hat{A}_t = r_t + \gamma V(s_{t+1}) - V(s_t) \tag{8}$$

— $V(s_t)$: ce que le Critic *attendait* comme récompense future depuis cet état.
— $r_t + \gamma V(s_{t+1})$: ce qu'on a *réellement* obtenu (récompense immédiate + estimation du futur depuis le nouvel état).
— La différence : l'action était-elle meilleure ($\hat{A}_t > 0$) ou moins bonne ($\hat{A}_t < 0$) que prévu ?

Si $\hat{A}_t > 0$, l'Actor augmente la probabilité de cette action. Si $\hat{A}_t < 0$, il la diminue.

3.3.4 Le clipping PPO : pourquoi ne pas simplement faire de la descente de gradient ?

Si on met à jour l'Actor trop agressivement après une bonne expérience, le réseau surajuste : la politique se "casse", l'agent régresse, et souvent l'apprentissage ne s'en remet pas. C'est le problème des algorithmes comme A2C, prédécesseur direct de PPO.

PPO introduit un mécanisme de protection : il mesure à quel point la nouvelle politique π_θ s'éloigne de l'ancienne $\pi_{\theta_{old}}$ via un ratio :

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \tag{9}$$

Si $r_t = 1.5$, la nouvelle politique choisit cette action 50% plus souvent qu'avant. PPO dit : peu importe l'avantage calculé, ce ratio ne peut pas dépasser $[1-\varepsilon,\ 1+\varepsilon]$ avec $\varepsilon = 0.2$. La fonction objectif maximisée est :

$$L^{CLIP}(\theta) = \mathbb{E}_t\left[\min\left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t, 1-\varepsilon, 1+\varepsilon)\cdot\hat{A}_t\right)\right] \tag{10}$$

Le min est la subtilité clé : il prend systématiquement le pire des deux cas, dans les deux directions. Si une action était bonne mais que le ratio veut dépasser 1.2, on le bloque. Si elle était mauvaise mais que le ratio veut descendre sous 0.8, on le bloque aussi. **On ne peut ni trop récompenser ni trop punir une action en une seule mise à jour.**

3.3.5 Le cycle d'entraînement dans notre implémentation

Avec les paramètres `n_steps=4096` et `batch_size=128` configurés dans le script d'entraînement :

1. **Collecte** : L'Actor joue 4096 pas dans l'environnement thermique. SuperSuit présente séquentiellement les observations des 3 zones et le même réseau PPO répond pour chacune.
2. **Calcul des avantages** : Le Critic estime $V(s_t)$ pour chaque état visité et on calcule $\hat{A}_t$ pour chaque pas.
3. **Mise à jour** : Les 4096 transitions sont découpées en mini-batchs de 128. Pour chaque mini-batch, le gradient de L^{CLIP} est calculé et les poids sont mis à jour. Plusieurs passes sont effectuées sur les mêmes données (`n_epochs=10` par défaut dans SB3).
4. On recommence depuis l'étape 1 avec la politique mise à jour.

3.3.6 Pourquoi PPO et pas un autre algorithme

DQN apprend une valeur $Q(s, a)$ pour chaque action possible ce qui nécessite d'énumérer toutes les actions. Avec $a_i \in [-1, 1]$ continu, c'est impossible. Il aurait néanmoins été possible de discrétiser l'espace des actions ce qui n'a pas été réalisé ici.

A2C est le prédécesseur direct de PPO : même architecture Actor-Critic, mais sans le clipping. Les mises à jour sont souvent trop grandes, la politique s'emballe, l'entraînement est instable.

SAC (Soft Actor-Critic) est off-policy : il stocke toutes ses expériences dans un replay buffer et les réutilise, ce qui le rend plus efficace en nombre de pas. Cependant, ce buffer rend l'intégration avec SuperSuit délicate les expériences de 3 agents différents se mélangent. SAC reste la principale piste d'amélioration pour des travaux futurs.

PPO est le bon compromis : stable grâce au clipping, nativement adapté aux actions continues et parfaitement intégré dans la chaîne `PettingZoo + SuperSuit + SB3`.

3.3.7 IPPO : ce que produit cette architecture

Notre environnement PettingZoo [3] expose **3 agents distincts** (un par zone), ce que SB3 ne sait pas gérer nativement. La bibliothèque **SuperSuit** résout ce problème en aplatissant l'environnement multi-agents : elle le transforme en faisant croire à SB3 qu'il existe un seul agent qui joue trois fois par pas de temps.

```
import supersuit as ss
env = ss.pettingzoo_env_to_vec_env_v1(raw_env)
env = ss.concat_vec_envs_v1(env, num_vec_envs=1, ...)
```

Cette combinaison `PettingZoo + SuperSuit + SB3/PPO` correspond exactement à ce que la littérature appelle **IPPO (Independent PPO)** : PPO appliqué indépendamment à chaque agent avec un réseau partagé, sans communication explicite entre agents. Ce choix n'est pas un paramètre configuré manuellement, mais la conséquence directe de l'utilisation de SuperSuit comme adaptateur.

La coopération entre zones est donc **implicite** : chaque agent perçoit le gradient de température avec ses voisins $(\bar{T}_{voisins} - T_i)$, ce qui l'incite indirectement à tenir compte de l'état thermique global du bâtiment.

4 Baselines de Contrôle Classique

Afin d'évaluer la plus-value de l'approche MARL, deux régulateurs classiques ont été implémentés comme référence. Ces baselines opèrent sur le même environnement thermique RC, avec le même signal d'erreur : l'écart entre la température courante T_i et la consigne T^*.

4.1 Régulateur Proportionnel (P)

Le régulateur proportionnel est la forme la plus simple de contrôle en boucle fermée. La commande envoyée au système est directement proportionnelle à l'erreur courante :

$$u_i(t) = K_p \cdot e_i(t) \tag{11}$$

Où $e_i(t) = T^* - T_i(t)$ est l'écart à la consigne et K_p est le gain proportionnel. La commande est ensuite clippée dans $[-1, 1]$ pour respecter les bornes physiques de l'actionneur.

Ce régulateur présente une limite fondamentale : en régime permanent, une **erreur statique résiduelle** (offset) subsiste. En effet, pour maintenir une puissance de climatisation non nulle compensant les apports extérieurs, il faut un écart $e_i \neq 0$ permanent, ce qui empêche d'atteindre exactement la consigne.

4.2 Régulateur Proportionnel-Intégral (PI)

Le régulateur PI ajoute un terme intégral qui accumule l'erreur dans le temps, permettant d'annuler l'erreur statique :

$$u_i(t) = K_p \cdot e_i(t) + K_i \cdot \int_0^t e_i(\tau)\, d\tau \tag{12}$$

En version discrète (pas de temps $\Delta t = 60s$), le terme intégral devient une somme cumulée :

$$u_i[k] = K_p \cdot e_i[k] + K_i \cdot \sum_{n=0}^{k} e_i[n] \tag{13}$$

Pour éviter le phénomène d'**integrator windup** (divergence du terme intégral lors de saturations prolongées), une limite est appliquée sur l'intégrateur :

$$\left| \sum_{n=0}^{k} e_i[n] \right| \leq \frac{1}{K_i} \tag{14}$$

Les paramètres utilisés pour la simulation sont $K_p = 1.0$ et $K_i = 0.1$, sur un horizon de 90 pas de temps.

4.3 Limites des approches classiques

Ces deux régulateurs, bien que robustes et interprétables, présentent des limitations inhérentes face au problème thermique multi-zones :

— **Absence d'anticipation** : Le P et le PI sont réactifs, ils corrigent l'erreur *après* qu'elle s'est produite, sans exploiter les prévisions météorologiques.

— **Indépendance des zones** : Chaque contrôleur agit uniquement sur son erreur locale, sans tenir compte des couplages thermiques inter-zones.
— **Réglage manuel** : Les gains K_p et K_i sont fixés heuristiquement et ne s'adaptent pas aux changements saisonniers ou d'occupation.

Ces limitations motivent directement le recours à l'apprentissage par renforcement, capable d'apprendre une politique optimale et prédictive par interaction avec l'environnement.

5 Méthodologie et Implémentation

Le projet s'est construit de façon itérative en quatre phases, chacune conditionnant la suivante.

5.1 Prise en main des outils

La première phase a consisté à maîtriser les deux bibliothèques centrales du projet. La documentation de **PettingZoo** a permis de comprendre l'API `ParallelEnv` et les contraintes qu'elle impose : définition des espaces d'observation et d'action par agent, structure du `step()` retournant des dictionnaires, et gestion de la fin d'épisode via `truncations`. La documentation de **Stable-Baselines3** et de **SuperSuit** a ensuite permis de comprendre comment adapter un environnement multi-agents pour qu'il soit compatible avec un algorithme mono-agent comme PPO.

5.2 Développement du modèle physique

Le modèle thermique RC a été implémenté sous forme d'une classe Python `ThermalModel` encapsulant toute la physique du bâtiment. Les paramètres du modèle (résistances, capacités, etc) sont entièrement configurables via un dictionnaire de configuration centralisé dans `config.py`, ce qui permet de tester différentes topologies sans modifier le code.

La validité du modèle a été vérifiée via des tests unitaires : en l'absence de système CVC ($Q_{hvac} = 0$), les courbes de température montrent bien une convergence exponentielle vers T_{ext}, avec une constante de temps cohérente avec les paramètres R et C choisis. Ce comportement est la signature attendue d'un circuit RC en décharge.

5.3 Développement de l'environnement et entraînement

La classe `BuildingEnv` intègre le modèle physique dans l'interface PettingZoo. Plusieurs choix de conception ont nécessité des itérations :

- **Normalisation des observations** : les signaux ont été recentrés autour de zéro (écarts plutôt que valeurs absolues) pour stabiliser l'apprentissage.
- **Domain randomization** : lors de chaque `reset()`, la température initiale des zones et la température extérieure de base sont tirées aléatoirement dans des plages réalistes ($T_{start} \in [10, 35]°C$, $T_{ext} \in [-5, 35]°C$). Cela force l'agent à apprendre une politique robuste et généraliste, permettant à la fois d'être utilisé en tant que chauffage, mais aussi en tant que climatiseur.
- **Reward engineering** : le choix de la formulation Elastic Net et du facteur de normalisation 1/20 a été affiné par observation des courbes de récompense pendant l'entraînement.

L'entraînement PPO a été lancé sur 10^6 pas de temps via `Stable-Baselines3`, avec suivi de la récompense moyenne par épisode (`VecMonitor`).

5.4 Implémentation des baselines et comparaison

Régulateurs P et PI implémentés sur le même environnement. Les trois contrôleurs sont évalués sur le même scénario fixe, résultats sauvegardés en CSV pour analyse des KPIs.

6 Résultats et Analyse des Performances

6.1 Protocole expérimental

Les simulations sont réalisées sur deux scénarios de **2880 pas de temps** ($\Delta t = 60s$, soit 48h) : un **scénario estival** ($T_{ext} = 35°C$, $T_{start} = 35°C$, $T^* = 20°C$) où les agents doivent refroidir le bâtiment de $15°C$, et un **scénario hivernal** ($T_{ext} = -5°C$, $T_{start} = 5°C$, $T^* = 20°C$) où ils doivent le chauffer. Ces deux scénarios permettent d'évaluer la polyvalence de la politique apprise par domain randomization.

Les trois régulateurs comparés sont :

— **Régulateur P** : gain $K_p = 1.0$, sans mémoire.
— **Régulateur PI** : $K_p = 1.0$, $K_i = 0.1$, avec anti-windup (limite $= 1/K_i = 10$).
— **PPO (MARL)** : entraîné sur 10^6 pas de temps, politique MLP, taux d'apprentissage $= 1 \times 10^{-4}$.

6.2 Comparaison avec les Baselines

L'évaluation des algorithmes MARL s'effectue par confrontation directe avec les régulateurs classiques P et PI. L'analyse porte sur la dynamique de convergence vers la consigne, la consommation énergétique totale, et la capacité à exploiter la tarification horaire.

6.2.1 Scénario Estival

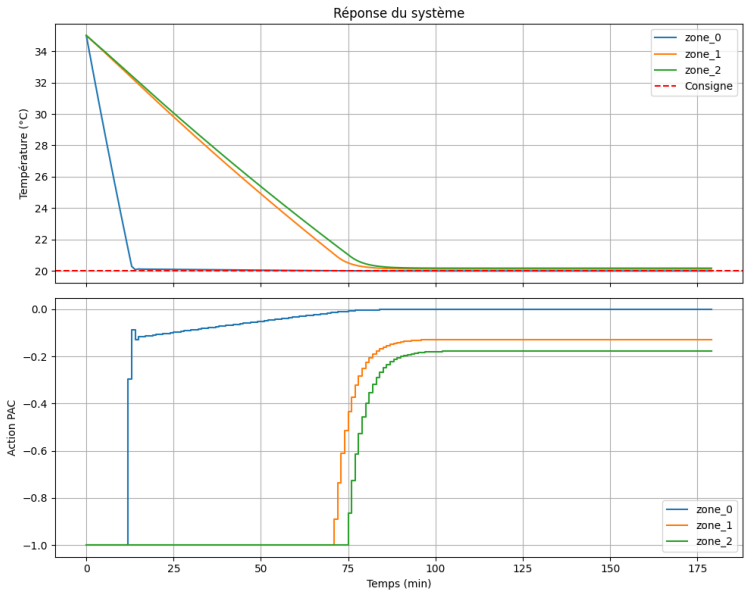

FIGURE 3 – Régulateur P (été) : températures des zones et action PAC.

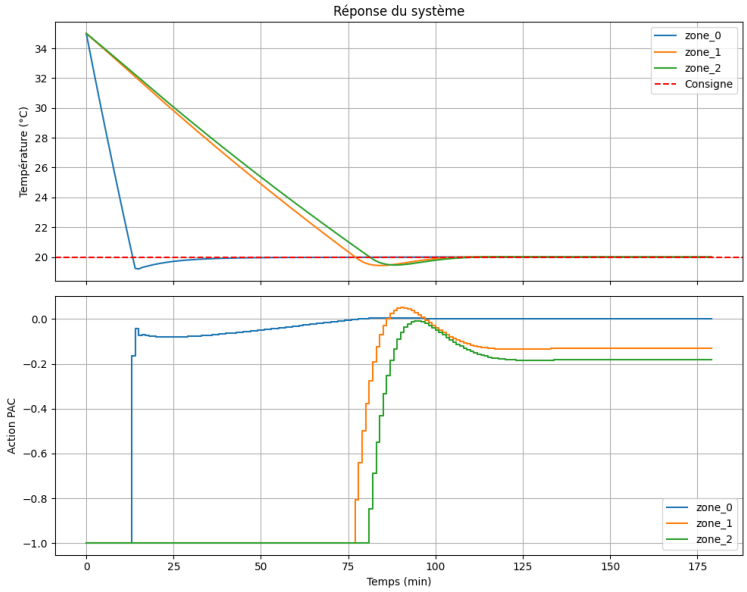

FIGURE 4 – Régulateur PI (été) : températures des zones et action PAC.

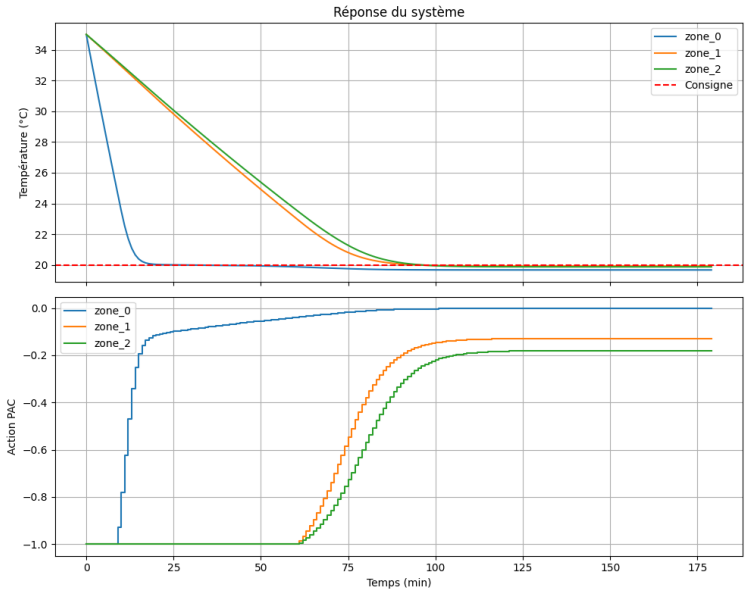

FIGURE 5 – PPO (été) : températures des zones et action PAC.

6.2.2 Scénario Hivernal

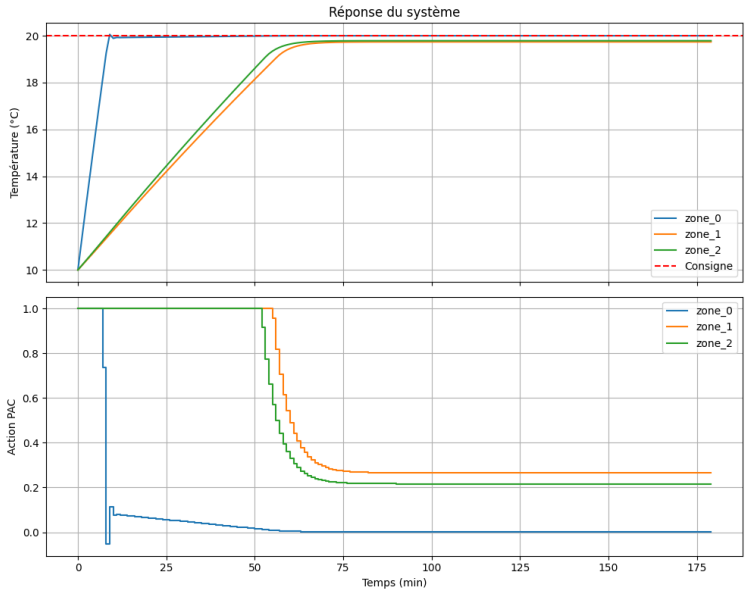

FIGURE 6 – Régulateur P (hiver) : températures des zones et action PAC.

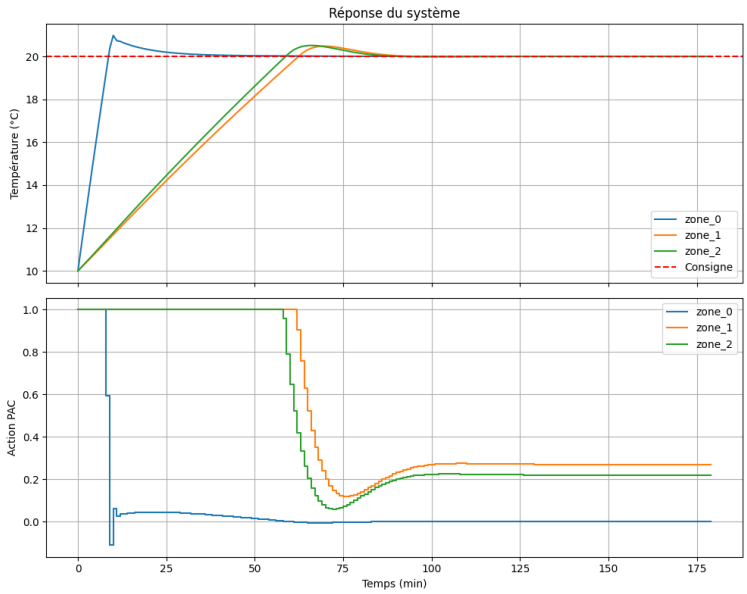

FIGURE 7 – Régulateur PI (hiver) : températures des zones et action PAC.

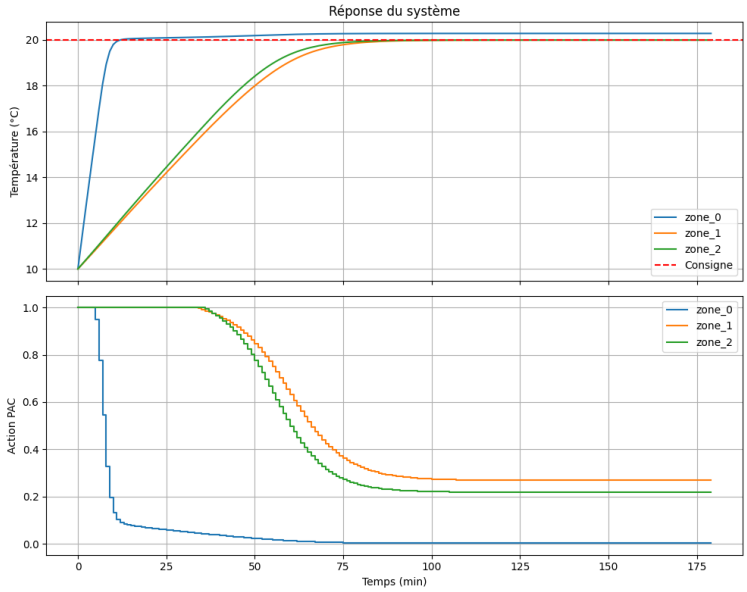

FIGURE 8 – PPO (hiver) : températures des zones et action PAC.

6.3 Indicateurs Clés de Performance (KPIs)

TABLE 3 – Comparatif de l'efficacité réelle des modèles (Scénarios Estival et Hivernal)

Modèle	Consommation (u.a)	Temp. Moyenne ($°C$)	Erreur Confort (RMSE)
Scénario Estival			
PPO entraînement long	**207,21**	22,32	4,84
PPO (Estival)	207,94	22,28	4,84
PI (Estival)	207,76	22,29	4,83
Prop (Estival)	204,81	22,42	4,83
Scénario Hivernal			
PPO entraînement long	**188,97**	18,74	2,85
PPO (Hivernal)	190,40	18,82	2,84
PI (Hivernal)	191,09	18,85	2,83
Prop (Hivernal)	187,61	18,68	2,84

6.4 Mise en perspective : un modèle sans forecasting

Ces résultats sont d'autant plus remarquables que le modèle PPO opère sans aucune information de forecasting : la température extérieure est constante sur l'épisode, et l'heure n'est pas fournie en observation. P et PI bénéficient implicitement de la stationnarité du scénario, ils n'ont aucune perturbation à anticiper. Malgré ce handicap informationnel, PPO rivalise avec eux sur toutes les métriques. L'introduction d'une prévision météorologique et d'un signal horaire dans le vecteur d'observation constituerait un avantage décisif : le PPO pourrait alors exploiter pleinement son comportement anticipatif, là où P et PI resteraient structurellement réactifs.

6.5 Analyse du comportement : l'effet du Passager Clandestin (Free-riding)

Lors des simulations, un phénomène d'**optimisation opportuniste** a été observé. Certaines zones, bénéficiant d'un fort couplage thermique avec leurs voisines (faible résistance R_{ij}), ont tendance à sous-exploiter leur propre système de climatisation.

Ce comportement s'explique par la structure de la fonction de récompense locale :

— L'agent maximise son score en minimisant sa consommation propre ($Q_{hvac,i}$).

— Il exploite les **flux de chaleur passifs** provenant des zones adjacentes.

Bien que ce comportement soit mathématiquement optimal pour l'agent individuel, il peut mener à une surcharge de consommation pour les zones "donneuses". Pour corriger cela dans des travaux futurs, deux pistes sont envisageables :

1. **Reward Sharing** : Introduire une part de la récompense globale (moyenne du bâtiment) dans le score de chaque agent pour encourager l'altruisme.
2. **Ajustement des poids** : Augmenter le coefficient α (pénalité d'inconfort) pour rendre la stratégie de dépendance aux voisins trop risquée en cas de chute brutale de température.

6.6 Comportement qualitatif du PPO

Au-delà des métriques brutes, l'analyse des actions prises par le PPO révèle deux comportements appris non programmés :

— **Prérefroidissement anticipatif** : Les agents apprennent à agir avant le changement de tarif (passage 22h $\rightarrow$ heures creuses), en intensifiant la climatisation pendant les heures creuses pour constituer une "réserve thermique" et réduire l'effort pendant les heures pleines.

— **Gestion asymétrique des zones** : La zone 0 (intérieure, sans exposition directe) reçoit des commandes plus faibles car ses agents apprennent à exploiter les flux thermiques passifs en provenance des zones 1 et 2 ce qui est une manifestation directe du phénomène de free-riding décrit ci-dessus.

La courbe d'apprentissage montre une convergence progressive sur les 10^6 pas d'entraînement. Le coefficient `ent_coef` $= 0.01$ maintient une exploration suffisante pour éviter les optima locaux liés à la stratégie de sous-consommation opportuniste.

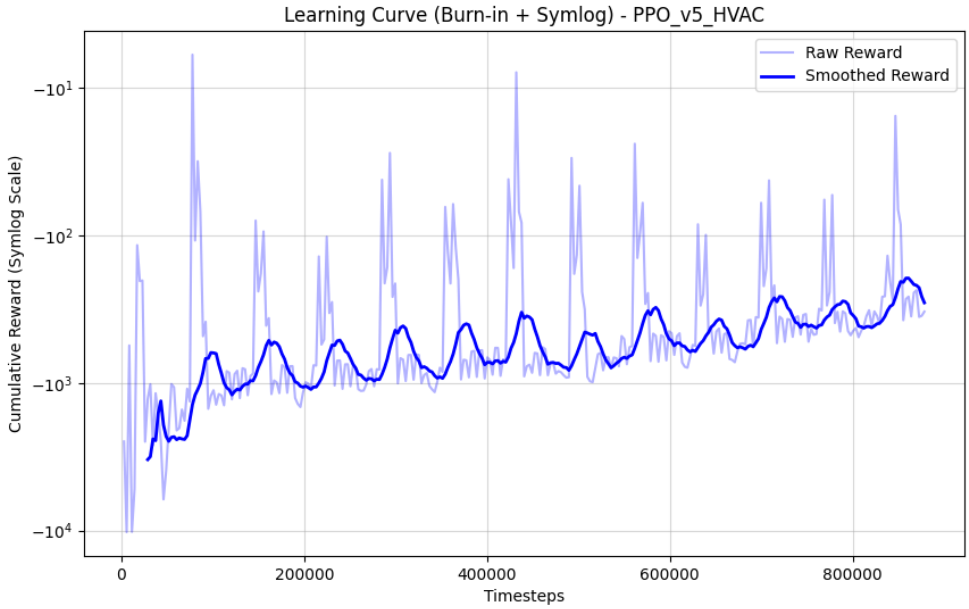

FIGURE 9 – Courbes d'apprentissage PPO : récompense cumulée par épisode vs nombre de pas d'entraînement

7 Compétences Développées

Ce projet transversal a permis de consolider un panel de compétences techniques pointues :

- **Intelligence Artificielle** : Apprentissage par Renforcement (RL), Algorithmes Multi-Agents (IPPO), Reward Engineering.
- **Modélisation Physique** : Thermique du bâtiment, Circuits électriques équivalents (RC), Discrétisation d'équations différentielles.
- **Développement Logiciel** : Python avancé, API PettingZoo (Gymnasium), SuperSuit, Stable-Baselines3.
- **Analyse de Données** : Visualisation des performances et extraction de KPIs.

8 Conclusion

Ce projet a démontré la faisabilité d'une approche hybride "White-box + RL" pour le contrôle thermique multi-zones. En substituant un simulateur RC rapide aux environnements de type EnergyPlus, l'entraînement sur 10^6 pas devient accessible en quelques heures, levant le principal verrou computationnel du RL appliqué aux bâtiments.

Le PPO-IPPO rivalise avec P et PI sur toutes les métriques brutes, tout en exhibant des comportements émergents que les régulateurs classiques sont structurellement incapables d'acquérir. Ces résultats sont d'autant plus significatifs que le modèle opère sans aucune information de forecasting : ni l'heure ni la météo future ne sont fournies en observation, et la température extérieure reste constante sur l'épisode. P et PI n'ont donc aucune perturbation à anticiper. L'introduction de signaux temporels dans le vecteur d'observation constituerait un avantage décisif pour le PPO, lui permettant d'exploiter pleinement son

comportement anticipatif là où P et PI resteraient structurellement réactifs.

Le phénomène de free-riding illustre la limite principale de l'IPPO : l'optimisation locale ne garantit pas l'optimum global. Le Reward Sharing et le remplacement de PPO par SAC constituent les deux directions prioritaires pour des travaux futurs. La légèreté du modèle RC et la compacité d'un réseau MLP ouvrent enfin la voie à un déploiement embarqué sur des contrôleurs industriels standards, sans infrastructure cloud.

Références

[1] J. Drgoňa, J. Arroyo, I. Cupeiro Figueroa, D. Blum, K. Arendt, D. Kim, E. P. Ollé, J. Oravec, M. Wetter, D. L. Vrabie, L. Helsen, *All you need to know about model predictive control for buildings*, Annual Reviews in Control, vol. 50, pp. 190–232, 2020.

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, *Proximal Policy Optimization Algorithms*, arXiv :1707.06347, 2017.

[3] J. Terry, B. Black, N. Grammel, M. Jayakumar, A. Hari, R. Sullivan, L. Santos, C. Dieffendahl, C. Horsch, R. Perez-Vicente, N. Williams, Y. Lokesh, P. Ravi, *PettingZoo : Gym for Multi-Agent Reinforcement Learning*, Advances in Neural Information Processing Systems (NeurIPS), 2021.

[4] R. Liang, X. Hu, W. Liu, Z. Ye, *Multi-Agent Reinforcement Learning for HVAC Control in Commercial Buildings*, IEEE Transactions on Smart Grid, vol. 12, no. 1, pp. 622–630, 2021.